

INTERVIEW DOCS

Coding Interview Notes

Patterns, Complexity Analysis & Java Snippets for Data Structure Problems

Java Agentic AI — Knowledge Base Reference

1. Big-O Complexity Cheat Sheet

Notation	Name	Example
$O(1)$	Constant	Array index access, HashMap get
$O(\log n)$	Logarithmic	Binary search, balanced BST operations
$O(n)$	Linear	Single pass through an array
$O(n \log n)$	Linearithmic	Efficient sorting (merge sort, heap sort)
$O(n^2)$	Quadratic	Nested loops, bubble sort
$O(2^n)$	Exponential	Naive recursive Fibonacci, subset generation

2. Common Problem-Solving Patterns

2.1 Two Pointers

Use two indices moving through a (usually sorted) array/string toward or away from each other, avoiding a nested loop and reducing $O(n^2)$ brute force to $O(n)$.

```
// Check if a sorted array has two numbers summing to target
boolean hasPairSum(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return true;
        else if (sum < target) left++;
        else right--;
    }
    return false;
}
```

2.2 Sliding Window

Maintain a window (contiguous subrange) that expands/shrinks over an array or string, useful for 'longest/shortest substring/subarray satisfying X' problems — avoids recomputation on every window shift.

```
// Longest substring without repeating characters
int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        while (window.contains(s.charAt(right))) {
            window.remove(s.charAt(left));
            left++;
        }
        window.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

```
}
```

2.3 Fast & Slow Pointers (Floyd's Cycle Detection)

```
boolean hasCycle(ListNode head) {  
    ListNode slow = head, fast = head;  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
        if (slow == fast) return true;    // pointers meet -> cycle exists  
    }  
    return false;  
}
```

2.4 BFS / DFS on Trees and Graphs

```
// BFS - level order traversal (uses a Queue)  
List<List<Integer>> levelOrder(TreeNode root) {  
    List<List<Integer>> result = new ArrayList<>();  
    if (root == null) return result;  
    Queue<TreeNode> queue = new LinkedList<>();  
    queue.offer(root);  
    while (!queue.isEmpty()) {  
        int size = queue.size();  
        List<Integer> level = new ArrayList<>();  
        for (int i = 0; i < size; i++) {  
            TreeNode node = queue.poll();  
            level.add(node.val);  
            if (node.left != null) queue.offer(node.left);  
            if (node.right != null) queue.offer(node.right);  
        }  
        result.add(level);  
    }  
    return result;  
}  
  
// DFS - recursive (uses the call stack)  
void dfs(TreeNode node, List<Integer> result) {  
    if (node == null) return;  
    result.add(node.val);  
    dfs(node.left, result);  
    dfs(node.right, result);  
}
```

2.5 Dynamic Programming

Break a problem into overlapping subproblems, solving each once and caching (memoizing) the result — applicable when a problem has both optimal substructure and overlapping subproblems.

```
// Fibonacci - top-down memoization  
Map<Integer, Long> memo = new HashMap<>();  
long fib(int n) {  
    if (n <= 1) return n;  
    if (memo.containsKey(n)) return memo.get(n);  
    long result = fib(n - 1) + fib(n - 2);  
}
```

```

        memo.put(n, result);
        return result;
    }

    // Bottom-up tabulation
    long fibIterative(int n) {
        if (n <= 1) return n;
        long[] dp = new long[n + 1];
        dp[1] = 1;
        for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
        return dp[n];
    }

```

2.6 Backtracking

```

// Generate all subsets
void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current));
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        backtrack(nums, i + 1, current, result); // explore
        current.remove(current.size() - 1);      // un-choose (backtrack)
    }
}

```

3. Sorting Algorithms Summary

Algorithm	Time (avg)	Space	Stable?
Bubble Sort	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$ avg, $O(n^2)$ worst	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(1)$	No

Note: A 'stable' sort preserves the relative order of equal elements — important when sorting by a secondary key after already sorting by a primary key.

4. Binary Search Template

```

int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // avoids integer overflow vs
        (left+right)/2
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

```

5. Useful Java Utilities for Interviews

```
// PriorityQueue as a min/max heap
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

// Deque as a stack or queue
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.pop();

// StringBuilder for efficient string building
StringBuilder sb = new StringBuilder();

// Arrays utility
Arrays.sort(arr);
int idx = Arrays.binarySearch(arr, target);

// HashMap for frequency counting
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) freq.merge(c, 1, Integer::sum);
```

6. Interview Approach Checklist

- Restate the problem in your own words and confirm constraints (input size, value ranges, edge cases).
- Walk through a small example by hand before coding.
- State the brute-force approach and its complexity first, then optimize.
- Think out loud — interviewers evaluate reasoning, not just the final answer.
- Consider edge cases explicitly: empty input, single element, duplicates, negative numbers, overflow.
- After coding, trace through your example again to verify correctness before declaring done.
- State the final time and space complexity clearly.